

Universidade Federal de Juiz de Fora
Departamento de Ciência da Computação
Especialização em Gerência de Projetos de Software na Era de Dados de
Sensores e IA

Plano de Qualidade para o Grafana

Camila Corrêa Vieira

Professor: Leonardo Reis

Trabalho final de Gerência da Qualidade de
Software, parte integrante da avaliação da dis-
ciplina.

Juiz de Fora

13 de Junho de 2026

1 O Software escolhido

1.1 Breve descrição

O projeto selecionado para a elaboração deste Plano de Qualidade é o Grafana, um sistema de código aberto (disponível em repositório público no GitHub) amplamente utilizado no mercado global para monitoramento, observabilidade e análise de dados interativa. Em sua arquitetura tecnológica, o sistema faz uso intensivo de bibliotecas reativas como o React no front-end, aliadas à segurança da tipagem estática do TypeScript. Em termos de infraestrutura, sua implantação e distribuição dependem fortemente de contêineres Docker. Além disso, o sistema atua primariamente realizando integrações de alta complexidade com potentes motores de busca e análise, como o Elasticsearch, e também possibilita a conexão com bancos orientados a grafos, como o Neo4j, para a extração de métricas de relacionamento complexas.

1.2 Justificativa da escolha

A escolha do Grafana justifica-se não apenas pela sua criticidade técnica e organizacional em escala global, mas também pela sua aplicação direta e fundamental no meu contexto de atuação, especificamente no projeto em que trabalho como desenvolvedora fullstack.

O Grafana atua como uma peça central na infraestrutura de observabilidade de milhares de organizações. Se o sistema de monitoramento falha, as equipes de operações ficam "cegas" para incidentes em seus próprios produtos. Devido à natureza colaborativa e massiva de seu desenvolvimento em repositórios Git, com milhares de contribuidores, torna-se necessária a aplicação de métodos rigorosos de engenharia de software e processos de verificação formal para evitar que regressões no código TypeScript ou configurações incorretas nos pipelines de CI/CD cheguem aos usuários finais.

Trazendo para o cenário prático do projeto, o Grafana é a ferramenta central que utilizo ativamente para consumir e visualizar os dados coletados pelo Prometheus no projeto. Essa arquitetura de observabilidade é vital para as nossas operações, sendo empregada principalmente para aferir o desempenho e garantir a otimização rigorosa das queries executadas no banco de dados em grafos Neo4j. A dependência técnica dessa visualização para assegurar a eficiência das consultas em grafos e a estabilidade do ecossistema provê a complexidade ideal e a vivência prática para a aplicação profunda dos conceitos de Qualidade de Produto e Processo exigidos nesta disciplina.

2 Identificação dos requisitos de qualidade

A definição dos atributos de qualidade fundamenta-se nas diretrizes do modelo de qualidade de produto estipulado pela norma ISO/IEC 25010. Considerando a criticidade do sistema para os modelos de negócios que o adotam, os requisitos foram detalhados a seguir:

- **Adequação Funcional:** O sistema deve ser capaz de interpretar consultas complexas (ex: queries textuais enviadas ao Elasticsearch) de forma exata. A integridade dos dados na tradução de resultados brutos para representações gráficas visuais é o núcleo do negócio.
- **Eficiência de Desempenho:** Sendo uma aplicação intensiva, a interface em React deve conseguir renderizar gráficos com dezenas de milhares de pontos de dados sem bloquear a main thread do navegador do usuário. Além disso, a latência de processamento interno deve ser minimizada.
- **Confiabilidade (Maturidade e Tolerância a Falhas):** Em cenários de instabilidade na fonte de dados, o sistema não deve sofrer falhas catastróficas (crashes). A arquitetura deve promover degradação graciosa, exibindo alertas descritivos.
- **Segurança:** Considerando o volume de dados sensíveis empresariais que transitam pelos painéis, o sistema exige práticas sólidas de Security by Design. É imperativo garantir o controle de acesso baseado em papéis (RBAC) e evitar vulnerabilidades na interface, como ataques XSS ou injeção de queries.
- **Manutenibilidade:** A utilização rigorosa do TypeScript e de padrões de projetos (ex: componentes bem delimitados no React) visa garantir um baixo nível de complexidade ciclomática. Isso permite que a vasta comunidade open-source identifique, teste e expanda funcionalidades de forma descentralizada.
- **Portabilidade:** A plataforma deve apresentar alta adaptabilidade. O encapsulamento através do Docker deve assegurar que a execução seja idêntica em ambientes de desenvolvimento local, servidores legados on-premis ou orquestradores modernos em nuvem.

3 Plano de qualidade de software

3.1 Objetivos da qualidade do projeto

Os objetivos organizacionais para o desenvolvimento e manutenção do produto são mensuráveis, visando a adoção do princípio de Gestão Quantitativa (Nível 4 do CMMI): Garantir disponibilidade de 99,9% (uptime) para instâncias baseadas em nuvem, assegurar que o tempo de resposta da interface para interações em componentes React não ultrapasse 2 segundos, manter a cobertura de testes automatizados unitários em um patamar mínimo de 85% para todo novo código submetido. e atingir conformidade absoluta com as diretrizes de codificação, não permitindo o merge de código no Git que contenha vulnerabilidades do tipo Critical apontadas por ferramentas de análise estática.

3.2 Estratégia de verificação e validação

A verificação garantirá que o produto está sendo construído corretamente segundo as especificações técnicas, enquanto a validação assegurará que o produto certo está sendo entregue ao usuário.

- Verificação Contínua: Uso extensivo de pipelines de CI (Integração Contínua) em cada Pull Request. Execução de linters de TypeScript, testes unitários e análises estáticas de segurança para avaliar a conformidade antes que o código atinja o repositório principal.
- Validação de Uso: Liberação progressiva de versões (releases alfa e beta) para comunidades de usuários restritas, aplicando testes de aceitação e validando a usabilidade e o valor analítico dos novos painéis integrados com bases de dados.

3.3 Papéis e responsabilidades da equipe

A qualidade não deve ser terceirizada apenas para analistas de testes; ela é sistêmica. Os papéis estão distribuídos da seguinte forma:

Papel	Responsabilidade na Qualidade
Engenheiros de Software (Devs)	Realizar revisões pessoais, aplicar <i>desk-checks</i> cruzados (<i>Code Review</i>), escrever testes unitários em TypeScript e garantir que as imagens Docker construídas sigam os preceitos do projeto.
Líder Técnico / Arquiteto	Conduzir as inspeções técnicas formais (revisões <u>arquiteturais</u>), gerir a redução do débito técnico e definir os padrões do <i>Design System</i> implementado em React.
Equipe de QA (Analistas de Qualidade)	Elaborar os cenários de automação E2E (<i>End-to-End</i>), monitorar as métricas de defeitos residuais e conduzir as auditorias de processos garantindo o <i>Quality Assurance</i> (foco na prevenção).
Gerente de Projeto / *Scrum Master*	Fomentar a cultura da melhoria contínua e viabilizar a análise das <u>Retrospectivas</u> do projeto, orientando as métricas para a gestão e facilitando a alocação de recursos.

Figura 1: Tabela de papéis

3.4 Critérios de aceitação do software

Para que um release seja considerado apto para produção, ele deve obrigatoriamente satisfazer os seguintes critérios (Definition of Done): Aprovação incondicional de todo o código fonte em um Pull Request por pelo menos dois engenheiros de software sêniores (Desk-check). Aprovação técnica na suíte de automação (100% Tempo de inicialização do contêiner Docker auditado com inicialização bem sucedida e sem avisos (warnings) fatais de memória. Verificação cruzada de segurança indicando zero anomalias de risco crítico ou alto inseridas na iteração atual.

Nome da Métrica	Descrição e Unidade de Medida	Forma de Coleta	Objetivo Estratégico
Densidade de Defeitos	Quantidade de erros por mil linhas de código (KLOC) ou por módulo/componente (Qtd/Módulo).	Extraída via ferramentas de análise estática e histórico de <i>issues</i> reportadas no Git.	Identificar pontos frágeis na arquitetura e sinalizar áreas que exigem refatoração pesada.
Cobertura de Testes (Coverage)	Percentual do código-fonte (em linhas ou <i>branches</i> lógicas) que é executado durante a suíte de testes automatizados (%).	Relatórios gerados por frameworks de teste de unidade na etapa de Integração Contínua.	Aumentar a confiança técnica antes dos deploys e evitar a introdução acidental de falhas.
Tempo Médio de Correção (MTTR)	O tempo médio exigido para diagnosticar e resolver uma falha após ela ser identificada em produção (Horas).	Calculado automaticamente pelas plataformas de controle de chamados e repositórios.	Avaliar a <u>manutenibilidade</u> do código e a eficiência das práticas de <i>troubleshooting</i> da equipe.
Índice de Retrabalho	O percentual do esforço do time gasto na <u>refação</u> ou correção de funcionalidades que já haviam sido "entregues" (%).	Apontamento de horas do time de engenharia classificado por tipo de tarefa.	Detectar processos falhos na fase de análise de requisitos ou falta de clareza nas documentações.
Tempo de Resposta do Sistema	Tempo transcorrido desde a solicitação de dados complexos até a renderização gráfica no lado do cliente (Segundos).	Métricas extraídas em tempo real por <i>*scripts*</i> sintéticos e telemetria nativa da aplicação.	Assegurar a adequação ao propósito em eficiência de desempenho exigida pela ISO/IEC 25010.

Figura 2: Métricas

3.5 Monitoramento e melhoria contínua

O plano será um artefato vivo, conduzido pelo ciclo de melhoria contínua. Reuniões mensais focadas na análise dos relatórios de métricas. A equipe deverá identificar gargalos de processo (fase Check) e implementar novos padrões corretivos (fase Act). E a cada final de ciclo de entrega (sprint), realizar uma reunião de retrospectiva para aprender com falhas sem a atribuição de culpa, elencando o que deve ser mantido, abandonado ou otimizado na rotina de testes e desenvolvimento.

4 Garantia da qualidade de software (Quality Assurance - QA)

Com base nos princípios de auditoria preventiva, a Garantia da Qualidade se concentra no processo, focando em "fazer da maneira certa" para reduzir defeitos endêmicos (abordagem proativa). Serão adotadas as seguintes práticas:

- Auditorias de Processo: Avaliação sistemática periódica para garantir que a equipe de engenharia segue fielmente a metodologia prescrita, utilizando ferramentas normatizadas como os lints (ex: ESLint para padronizar o TypeScript do projeto). O time atua como a consciência crítica, checando a aderência aos pipelines de desenvolvimento.
- Revisões de Artefatos Gerenciais: Documentos de especificação de requisitos e designs arquitetônicos deverão passar por revisões técnicas baseadas em checklists (verificação informal) e Walk-throughs colaborativos. Isso mitiga ambiguidades lógicas antes que uma única linha de código seja produzida.
- Gestão de Configuração e Mudança: Implementação estrita do controle de versão utilizando Git. Todo artefato de infraestrutura, como configurações dos contêineres Docker (Dockerfiles) e manifests, é tratado como código, garantindo que o ciclo de evolução possua histórico impecável e permita reversões (rollbacks) consistentes e imediatas.

5 Controle da qualidade de software (Quality Control - QC)

Em contraste com a Garantia da Qualidade, o Controle de Qualidade atua diretamente no produto (foco na detecção). O produto passará por Testes Unitários para atestar o

funcionamento atômico de funções React e helpers TypeScript. Seguirão os Testes de Integração, cuja relevância é central, validando a sinergia entre o software e instâncias efêmeras de bancos de dados levantados (Elasticsearch / Neo4j) via infraestrutura Docker no CI. Finalmente, Testes de Aceitação validarão a entrega contra os requisitos de negócio estipulados. Para áreas sensíveis do sistema (módulos de segurança ou acesso a dados confidenciais), o simples desk-check será substituído por inspeções técnicas rigorosas. Nessas reuniões, membros analisam o código usando checklists históricas que isolam os padrões de falhas comuns da equipe. Uso integrado de motores de verificação analítica acoplados no servidor para mapear dependências desatualizadas, localizar vulnerabilidades lógicas e monitorar infrações aos padrões de manutenibilidade da ferramenta sem executar a aplicação ativamente.

6 Conclusão

A implementação sistêmica deste Plano de Qualidade de Software alinha o esforço técnico de desenvolvimento aos objetivos empresariais mais críticos. Ao integrar as visões de Garantia e Controle da Qualidade ao longo de todo o ciclo de vida do software, baseando-se em modelos de maturidade consolidados como o CMMI e no regramento do modelo ISO/IEC, a organização não apenas reduz substancialmente seus custos operacionais de retrabalho e correções tardias, como também fomenta a competitividade. Através do monitoramento quantitativo e da dedicação preventiva guiada pelo ciclo PDCA, a arquitetura torna-se mais resiliente, segura e excepcionalmente adaptada para entregar excelência contínua aos usuários finais.